Métodos Numéricos

Prof. Jonatha Costa

2026

Organização

1. Ajustes de Curvas
   - Conceitos
2. Regressão Linear - mínimos quadrados
   - Implementação computacional
3. Análise de erros
   - Métricas de análise
4. Linearização
   - Conceitos
   - Implementação computacional
5. Regressão Polinomial
   - Implementação computacional
   - Implementação computacional
6. Polinômio interpolador de Lagrange
   - Implementação computacional
7. Polinômio interpolador de Newton
   - Implementação computacional
8. Splines: introdução
   - Splines Linear
     - Implementação computacional
   - Splines Quadráticas
     - Implementação computacional
   - Splines Cúbicas
     - Implementação computacional

MN aplicados à Engenharia

Objetivo da aula

- Apresentar conteúdo de Ajuste de Curvas
  - Interpolação e extrapolação
  - Regressão Linear por Mínimos Quadrados
  - Linearização de Equações não lineares
  - Polinômio de Lagrange e de Newton
  - Spline linear, quadrática e cúbica

MN aplicados à Engenharia

Considerações basilares de engenharia e modelagem matemática

1. Em Engenharia, grandezas físicas são frequentemente obtidas por meio de **medições experimentais**, realizadas por sensores e sistemas de aquisição de dados;
2. As medições são armazenadas como um conjunto de **dados discretos**, constituindo os chamados **dados ou pontos experimentais**;
3. A representação discreta fornece apenas uma quantidade finita de informações sobre o fenômeno observado, podendo não reproduzir integralmente o comportamento contínuo da grandeza;
4. A partir dos pontos experimentais, busca-se obter uma **função matemática** capaz de representar, de forma adequada, o comportamento observado;
5. Essa representação é obtida por meio do **ajuste de curvas**, no qual os parâmetros de uma função são determinados de modo a minimizar uma medida de erro entre o modelo e os dados experimentais;
6. Dependendo do problema, podem ser utilizadas diferentes funções de aproximação, incluindo **polinômios de diferentes ordens**.

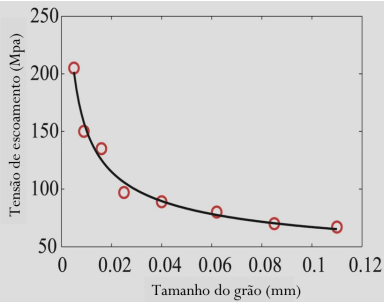
MN aplicados à Engenharia

Ajuste de Curvas e interpolação

Definição

- **Interpolação**: procedimento utilizado para **estimar valores entre pontos experimentais conhecidos**, dentro do intervalo dos dados disponíveis.
- **Extrapolação**: procedimento utilizado para **estimar valores fora do intervalo** dos dados experimentais, utilizando o comportamento identificado no modelo.
- **Ajuste de curvas**: procedimento utilizado para determinar os **parâmetros de uma função matemática** que represente adequadamente um conjunto de dados, segundo um determinado **critério de erro**.

Figura 1: Tamanho de grão × tensão



Fonte: GILAT,(2008)

Perceba na Figura 1 os pontos discretos de uma medição que pode ser expressa pela Equação 1.

$$\sigma = Cd^m \tag{1}$$

em que:

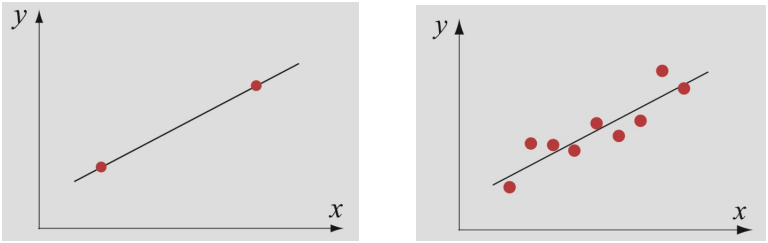
- σ - Tensão de escoamento
- C - é uma constante
- d - é o tamanho do grão
- m - é o grau da equação

MN aplicados à Engenharia

Ajuste de curvas – Polinômio de 1º grau:

- É uma das formas mais simples de representar matematicamente a relação entre duas variáveis a partir de dados experimentais;
- O modelo é definido por $y = a_1x + a_0$, em que a_1 representa a inclinação da reta e a_0 o intercepto com o eixo y;
- Considere a Figura 2.
  - Para **dois pontos**, existe uma única reta que passa exatamente pelos pontos experimentais;
  - Para **vários pontos**, os dados podem não estar sobre uma única reta. Nesse caso, a_0 e a_1 são determinados de modo que a reta forneça a melhor aproximação possível aos dados, segundo um determinado critério de erro.

Figura 2: Curvas



(a) Dois pontos (b) Vários pontos

Fonte: GILAT,(2008)

MN aplicados à Engenharia

Ajuste de Curvas e Interpolação

Qualidade do ajuste de curvas

- Para avaliar o quanto uma função representa um conjunto de dados experimentais, é necessário definir uma **medida de erro** entre os valores observados e os valores estimados pelo modelo;
- O **resíduo** representa a diferença entre o valor experimental y_i e o valor estimado pela função $f(x_i)$;
- Para um conjunto de n pontos experimentais, cada ponto possui um resíduo associado;
- Quanto menores forem os resíduos, menor tende a ser a diferença entre o modelo matemático e os dados experimentais.

Valor experimental	**Valor estimado**
y_i	$\hat{y}_i = f(x_i)$

Resíduo do i-ésimo elemento de y pode ser expresso pela Equação 2.

$$\boxed{r_i = y_i - f(x_i)} \tag{2}$$

MN aplicados à Engenharia

Ajuste de Curvas e Interpolação

Melhor ajuste – Método dos mínimos quadrados

- Note que os resíduos podem assumir valores positivos ou negativos. Portanto, a simples soma dos resíduos pode resultar em cancelamentos;
- Para evitar esse efeito, utiliza-se a **soma dos quadrados dos resíduos** como medida global do erro;
- Para um polinômio de primeiro grau: $f(x) = a_1 x + a_0$, o erro quadrático global é definido pela Equação 3.

$$E(a_0, a_1) = \sum_{i=1}^{n} r_i^2 = \sum_{i=1}^{n} [y_i - (a_1 x_i + a_0)]^2 \tag{3}$$

Critério de mínimos quadrados expresso na Equação 4.

$$\min_{a_0, a_1} E(a_0, a_1) = \min_{a_0, a_1} \sum_{i=1}^{n} [y_i - (a_1 x_i + a_0)]^2 \tag{4}$$

O melhor ajuste é aquele que **minimiza a soma dos quadrados dos resíduos**.

Como $r_i^2 \geq 0$, resíduos positivos e negativos não se cancelam.

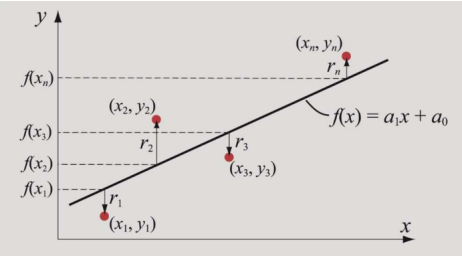
MN aplicados à Engenharia

Ajuste de Curvas e Interpolação

Melhor ajuste – Método dos mínimos quadrados

- Considere a Figura 3. Note que os resíduos podem assumir valores positivos ou negativos. A simples soma dos resíduos pode, portanto, produzir cancelamentos;
- Para evitar esse efeito, utiliza-se a **soma dos quadrados dos resíduos** como medida global do erro;
- Para um polinômio de primeiro grau: $f(x) = a_1 x + a_0$, o erro quadrático global é expresso na Equação 5.

Figura 3: Polinômio de grau 1



Fonte: Gilat,(2008).

Erro quadrático global

$$E(a_0, a_1) = \sum_{i=1}^{n} r_i^2 \qquad (5)$$

Reescrita como: $\sum_{i=1}^{n} [y_i - (a_1 x_i + a_0)]^2$

$r_i^2 \geq 0 \Rightarrow$ não há cancelamento entre resíduos positivos e negativos.

Organização

1. Ajustes de Curvas
   - Conceitos
2. Regressão Linear - mínimos quadrados
   - Implementação computacional
3. Análise de erros
   - Métricas de análise
4. Linearização
   - Conceitos
   - Implementação computacional
5. Regressão Polinomial
   - Implementação computacional
   - Implementação computacional
6. Polinômio interpolador de Lagrange
   - Implementação computacional
7. Polinômio interpolador de Newton
   - Implementação computacional
8. Splines: introdução
   - Splines Linear
     - Implementação computacional
   - Splines Quadráticas
     - Implementação computacional
   - Splines Cúbicas
     - Implementação computacional

MN aplicados à Engenharia

Regressão Linear pelo método dos Mínimos Quadrados

- A minimização da soma dos quadrados dos resíduos conduz a um sistema de duas equações, obtido pela anulação das derivadas parciais de E em relação a a_0 e a_1.
- Portanto, o melhor ajuste é obtido quando o erro global, definido pela equação a seguir, é mínimo:

$$\min_{a_0,a_1} E(a_0,a_1) = \min_{a_0,a_1} \sum_{i=1}^{n} [y_i - (a_1 x_i + a_0)]^2$$

- Derivando o erro E em função dos coeficientes a_0 e a_1, tem-se:

$$\frac{\partial E}{\partial a_0} = -2 \sum_{i=1}^{n} \Big[y_i - a_i x_i - a_0 \Big] = 0$$

$$\frac{\partial E}{\partial a_1} = -2 \sum_{i=1}^{n} \Big[y_i - a_i x_i - a_0 \Big] x_i = 0$$

MN aplicados à Engenharia

Regressão Linear pelo método dos Mínimos Quadrados

- Resolvendo as derivadas, tem-se que:

$$a_1 = \frac{n\sum_{i=1}^{n} x_i y_i - \left(\sum_{i=1}^{n} x_i\right)\left(\sum_{i=1}^{n} y_i\right)}{n\sum_{i=1}^{n} x_i^2 - \left(\sum_{i=1}^{n} x_i\right)^2}$$

$$a_0 = \frac{\left(\sum_{i=1}^{n} x_i^2\right)\left(\sum_{i=1}^{n} y_i\right) - \left(\sum_{i=1}^{n} x_i y_i\right)\left(\sum_{i=1}^{n} x_i\right)}{n\sum_{i=1}^{n} x_i^2 - \left(\sum_{i=1}^{n} x_i\right)^2}$$

- Por conveniência pode-se expressar o somatório como S_k:

$$\boxed{S_x = \sum_{i=1}^{n} x_i}, \quad \boxed{S_y = \sum_{i=1}^{n} y_i}, \quad \boxed{S_{xy} = \sum_{i=1}^{n} x_i y_i}, \quad \boxed{S_{xx} = \sum_{i=1}^{n} x_i^2}.$$

Portanto, os termos a_0 e a_1 de $f(x) = a_1 x + a_0$, são: $\boxed{a_1 = \frac{nS_{xy} - S_x S_y}{nS_{xx} - (S_x)^2}}$, $\boxed{a_0 = \frac{S_{xy} S_y - S_{xy} S_x}{nS_{xx} - (S_x)^2}}$

MN aplicados à Engenharia

Regressão Linear em Python

- O código a seguir utiliza a estruturação anteriormente detalhada.

Cálculo dos coeficientes

```
import numpy as np
import matplotlib.pyplot as plt
x = np.arange(1, 9)
y = np.array([2.1, 3.9, 4.2, 6.1,
7.5, 8.3, 8.9, 10.4])

def reglin(x, y):
    if len(x) != len(y):
            return print('Falha!')
    n = len(x)
    Sx, Sy = sum(x), sum(y)
    Sxy, Sxx = sum(x*y), sum(x**2)
    a1 = (n*Sxy - Sx*Sy)/(n*Sxx - Sx**2)
    a0 = (Sxx*Sy - Sxy*Sx)/(n*Sxx - Sx**2)
    return a1, a0
a1, a0 = np.round(reglin(x, y), 4)
print(f"p(x) = {a1}*x + {a0}")
```

Visualização da regressão

```
def graf_reglin(x, y):
    a1, a0 = reglin(x, y)

    print("Coef. angular:", a1)
    print("Coef. linear:", a0)

    plt.scatter(x, y, label="Dados")
    plt.plot(x, a0 + a1*x, 'r-',
    label="Regressao linear")
    plt.xlabel("x")
    plt.ylabel("y")
    plt.legend()
    plt.grid()
    plt.show()

graf_reglin(x, y)
```

MN aplicados à Engenharia

Regressão Linear em Python

- O código a seguir utiliza o comando `np.polyfit(x,y,1)` da biblioteca `NumPy` para calcular os coeficientes da reta de regressão linear pelo método dos mínimos quadrados.

Código Python

```
import numpy as np                                  # Biblioteca de operacoes numericas
import matplotlib.pyplot as plt                     # Biblioteca de graficos
x = np.arange(1, 9)                                 # Dados de entrada
y = np.array([2.1, 3.9, 4.2, 6.1,                   # Dados observados
7.5, 8.3, 8.9, 10.4])
a1, a0 = np.polyfit(x, y, 1)                        # Calcula os coeficientes a1 e a0
print("Coef. angular:", a1)                         # Exibe o coeficiente angular
print("Coef. linear:", a0)                          # Exibe o coeficiente linear
plt.scatter(x, y, label="Dados")                    # Traca os dados observados
plt.plot(x, a0 + a1*x, 'r-',                        # Traca a reta ajustada
label="Regressao linear")
plt.xlabel("x")                                     # Define o rotulo do eixo x
plt.ylabel("y")                                     # Define o rotulo do eixo y
plt.legend()                                        # Exibe a legenda do grafico
plt.grid()                                          # Adiciona a grade ao grafico
plt.show()                                          # Exibe o grafico
```

MN aplicados à Engenharia

Ajuste de Curvas e interpolação

Exercícios de Fixação — Regressão Linear (I)

Considere os dados experimentais a seguir:

$$X = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12\}$$

$$Y = \{2.1, 3.9, 5.2, 7.1, 8.0, 10.2, 11.1, 13.2, 14.1, 16.2, 17.0, 19.1\}$$

1. Determine os coeficientes a_0 e a_1 pelo método dos mínimos quadrados e obtenha a equação da reta ajustada.
2. Implemente a regressão linear em Python utilizando `np.polyfit()`. Apresente os coeficientes calculados e construa o gráfico dos dados experimentais e da reta ajustada.
3. Utilize a reta obtida para estimar y quando $x = 10.5$ e $x = 15$. Classifique essa estimativa como interpolação ou extrapolação e justifique sua resposta.

MN aplicados à Engenharia

Ajuste de Curvas e Interpolação

Exercícios de Fixação — Regressão Linear (II)

1. **Ajuste linear de dados experimentais**

   Em um experimento de Engenharia, foram obtidos os seguintes dados, relacionando a temperatura x (°C) e uma grandeza física medida y:

   $$X = \{10, 20, 30, 40, 50, 60\}$$

   $$Y = \{12.1, 18.3, 25.8, 31.2, 39.1, 44.7\}$$

   a) Implemente em Python o ajuste linear $\hat{y} = a_1 x + a_0$ utilizando `np.polyfit()`.
   b) Construa um gráfico contendo os dados experimentais e a reta ajustada.
   c) Calcule o coeficiente de determinação R^2 e interprete a qualidade do ajuste.

2. **Interpolação e extrapolação**

   Utilizando o modelo ajustado no Exercício 1:

   a) Estime y para $x = 35$ e $x = 75$.
   b) Classifique cada estimativa como interpolação ou extrapolação, justificando sua resposta.
   c) Discuta por que a confiabilidade das estimativas pode diminuir fora do intervalo experimental.

MN aplicados à Engenharia

Ajuste de Curvas e Interpolação

Exercícios de Fixação — Regressão Linear (III)

3. **Comparação entre ajustes lineares**

   Um ensaio de laboratório produziu os seguintes resultados para a relação entre o tempo x (s) e a distância percorrida y (m):

$$X = \{1, 2, 3, 4, 5, 6, 7\}$$

$$Y = \{2.8, 5.1, 7.9, 8.8, 11.2, 13.1, 16.0\}$$

   Implemente um programa em Python que:

   1. Determine a reta de regressão linear pelo método dos mínimos quadrados utilizando `np.polyfit()`.
   2. Calcule os valores estimados $\hat{y}_i$ e os resíduos $e_i = y_i - \hat{y}_i$.
   3. Construa um gráfico dos dados e da reta ajustada e outro gráfico dos resíduos em função de x.

   d) Calcule o erro quadrático médio (EQM) e o coeficiente de determinação R^2.

   4. Analise o gráfico dos resíduos e discuta se o modelo linear é adequado para representar os dados experimentais.

MN aplicados à Engenharia

Curiosidade: Métodos de Regressão Linear

Há somente um método de regressão linear?

MN aplicados à Engenharia

Curiosidade: Métodos de Regressão Linear

Há somente um método de regressão linear?

- **Mínimos Quadrados Ordinários (MQO):** minimiza os resíduos quadráticos. $\min_{\boldsymbol{\beta}} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$
- **Mínimos Quadrados Ponderados (MQP):** atribui pesos às observações. $\min_{\boldsymbol{\beta}} \sum_{i=1}^{n} w_i (y_i - \hat{y}_i)^2$
- **Mínimos Quadrados Generalizados (MQG):** considera a covariância dos erros. $\min_{\boldsymbol{\beta}} (\mathbf{y} - \mathbf{X}\beta)^T \boldsymbol{\Sigma}^{-1} (\mathbf{y} - \mathbf{X}\beta)$
- **Regressão Linear Múltipla:** utiliza várias variáveis explicativas. $\hat{y} = a_0 + a_1 x_1 + \cdots + a_p x_p$
- **Regressão Regularizada (Ridge e Lasso):** penaliza os coeficientes para controlar a complexidade.

  Ridge: $J + \lambda \sum_{j=1}^{p} a_j^2$ Lasso: $J + \lambda \sum_{j=1}^{p} |a_j|$
- **Gradiente Descendente:** atualiza os coeficientes iterativamente. $\boldsymbol{\beta}^{(k+1)} = \boldsymbol{\beta}^{(k)} - \alpha \nabla J(\boldsymbol{\beta}^{(k)})$
- **Regressão Robusta:** reduz a influência de valores atípicos. $\min_{\boldsymbol{\beta}} \sum_{i=1}^{n} \rho(y_i - \hat{y}_i)$

Pesquise sobre os demais métodos distintos acima, visando comparar a performance de implementação computacional!

Organização

1. Ajustes de Curvas
   - Conceitos
2. Regressão Linear - mínimos quadrados
   - Implementação computacional
3. Análise de erros
   - Métricas de análise
4. Linearização
   - Conceitos
   - Implementação computacional
5. Regressão Polinomial
   - Implementação computacional
   - Implementação computacional
6. Polinômio interpolador de Lagrange
   - Implementação computacional
7. Polinômio interpolador de Newton
   - Implementação computacional
8. Splines: introdução
   - Splines Linear
     - Implementação computacional
   - Splines Quadráticas
     - Implementação computacional
   - Splines Cúbicas
     - Implementação computacional

Métricas de Erro: MAE, MSE e RMSE

Definição matemática

MAE

$$\mathrm{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

MSE

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

RMSE

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}$$

Implementação em Python

```python
import numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error
def mae(y_true, y_pred):
    return mean_absolute_error(y_true, y_pred)
def mse(y_true, y_pred):
    return mean_squared_error(y_true, y_pred)
def rmse(y_true, y_pred):
    return np.sqrt(mse(y_true, y_pred))
```

- **MAE:** média dos módulos dos erros.
- **MSE:** média dos erros ao quadrado.
- **RMSE:** raiz do MSE, na unidade da variável prevista.

Métricas Percentuais e de Ajuste

Definição matemática

MAPE

$$\text{MAPE} = \frac{100}{N} \sum_{i=1}^{N} \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

Coeficiente R^2

$$R^2 = 1 - \frac{\sum_{i=1}^{N} (y_i - \hat{y}_i)^2}{\sum_{i=1}^{N} (y_i - \bar{y})^2}$$

Bias

$$\text{Bias} = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)$$

Implementação em Python

```python
import numpy as np
from sklearn.metrics import r2_score
def mape(y_true, y_pred):
    return np.mean(np.abs((y_true - y_pred) / y_true)) * 100
def r2(y_true, y_pred):
    return r2_score(y_true, y_pred)
def bias(y_true, y_pred):
    return np.mean(y_pred - y_true)
```

- **MAPE:** erro absoluto percentual médio.
- **R^2:** proporção da variabilidade explicada pelo modelo, em relação à média dos valores reais.
- **Bias:** viés médio; positivo indica superestimação e negativo indica subestimação.

Métricas Complementares e Resultados

Definição matemática

MSLE – Erro Logarítmico Quadrático Médio

$$\text{MSLE} = \frac{1}{N}\sum_{i=1}^{N}[\ln(1+y_i) - \ln(1+\hat{y}_i)]^2$$

WMSE – Erro Quadrático Médio Ponderado

$$\text{WMSE} = \frac{\sum_{i=1}^{N} w_i (y_i - \hat{y}_i)^2}{\sum_{i=1}^{N} w_i}$$

Implementação em Python

```python
import numpy as np
from sklearn.metrics import mean_squared_error
def msle(y_true, y_pred):
    return mean_squared_error(np.log1p(y_true), np.log1p(y_pred))
def wmse(y_true, y_pred, weights=None):
    if weights is None:
    weights = np.array([0.1, 0.2, 0.3, 0.4])
    return np.sum(weights * (y_true-y_pred)**2) / np.sum(weights)
if __name__ == "__main__":
    y_true = np.array([3.0, -0.5, 2.0, 7.0])
    y_pred = np.array([2.5,  0.0, 2.0, 8.0])
    print(f"MSLE: {msle(y_true, y_pred):.4f}")
    print(f"WMSE: {wmse(y_true, y_pred):.4f}")
```

- **MSLE:** avalia erros no domínio logarítmico, reduzindo a influência de diferenças absolutas grandes.
- **WMSE:** permite atribuir maior importância a observações específicas por meio dos pesos w_i.

Organização

1. Ajustes de Curvas
   - Conceitos
2. Regressão Linear - mínimos quadrados
   - Implementação computacional
3. Análise de erros
   - Métricas de análise
4. Linearização
   - Conceitos
   - Implementação computacional
5. Regressão Polinomial
   - Implementação computacional
   - Implementação computacional
6. Polinômio interpolador de Lagrange
   - Implementação computacional
7. Polinômio interpolador de Newton
   - Implementação computacional
8. Splines: introdução
   - Splines Linear
     - Implementação computacional
   - Splines Quadráticas
     - Implementação computacional
   - Splines Cúbicas
     - Implementação computacional

MN aplicados à Engenharia

Linearização de Equações

- Muitas situações na ciência e na engenharia mostram que a relação entre as grandezas envolvidas **não é linear**;
- Entretanto, funções não-lineares que podem ser escritas em uma forma tal que possibilite a determinação dos coeficientes que levam ao melhor ajuste com o emprego do método da regressão linear por mínimos quadrados;
  - $y = b \cdot x^m$
- Para que a regressão linear possa ser utilizada, a equação não-linear de duas variáveis deve ser modificada de tal forma que a nova equação seja linear com termos contendo as variáveis originais. Dessa forma, para a equação acima tem-se:

$$\boxed{y = b \cdot x^m \rightarrow ln(y) = ln(b \cdot x^m) = m \cdot ln(x) + ln(b)}$$

  - equação linear em termos de $ln(x)$ tal que a equação está na forma $Y = a_1 X + a_0$, em que $Y = ln(y)$, $a_1 = m$,$X = ln(x)$ e $a_0 = ln(b)$.
  - Isso significa que uma regressão linear por mínimos quadrados pode ser usada para fazer com que uma equação na forma $y = b \cdot x^m$ se ajuste a um conjunto de pontos x_i, y_i.

Linearização de Equações Não Lineares

Na Tabela 1 são apresentadas algumas equações não lineares expressas em formas lineares no formato $Y = a_1 X + a_0$.

Tabela 1: Equações não lineares na forma linear.

Equação não linear	Forma linear	Relação com $Y = a_1 X + a_0$	Valores para a regressão linear por mínimos quadrados	Gráficos onde os dados medidos parecem se ajustar a uma linha reta
$y = bx^m$	$\ln(y) = m\ln(x) + \ln(b)$	$Y = \ln(y), \quad X = \ln(x)$ $a_1 = m, \quad a_0 = \ln(b)$	$\ln(x_i)$ e $\ln(y_i)$	Gráfico y vs. x em eixos x e y logarítmicos. Gráfico $\ln(y)$ vs. $\ln(x)$ em eixos lineares.
$y = be^{mx}$	$\ln(y) = mx + \ln(b)$	$Y = \ln(y), \quad X = x$ $a_1 = m, \quad a_0 = \ln(b)$	x_i e $\ln(y_i)$	Gráfico y vs. x com eixo x linear e eixo y logarítmico. Gráfico $\ln(y)$ vs. x em eixos lineares.
$y = b10^{mx}$	$\log(y) = mx + \log(b)$	$Y = \log(y), \quad X = x$ $a_1 = m, \quad a_0 = \log(b)$	x_i e $\log(y_i)$	Gráfico y vs. x com eixo x linear e eixo y logarítmico. Gráfico $\log(y)$ vs. x em eixos lineares.
$y = \dfrac{1}{mx + b}$	$\dfrac{1}{y} = mx + b$	$Y = \dfrac{1}{y}, \quad X = x$ $a_1 = m, \quad a_0 = b$	x_i e $\dfrac{1}{y_i}$	Gráfico $1/y$ vs. x em eixos lineares.
$y = \dfrac{mx}{b + x}$	$\dfrac{1}{y} = \dfrac{b}{m}\dfrac{1}{x} + \dfrac{1}{m}$	$Y = \dfrac{1}{y}, \quad X = \dfrac{1}{x}$ $a_1 = \dfrac{b}{m}, \quad a_0 = \dfrac{1}{m}$	$\dfrac{1}{x_i}$ e $\dfrac{1}{y_i}$	Gráfico $1/y$ vs. $1/x$ em eixos lineares.

Nota: x_i e y_i representam os dados medidos. Para as linearizações logarítmicas, exige-se $x_i > 0$ e $y_i > 0$.

MN Aplicados à Engenharia

Linearização de Equações Não Lineares

A escolha de um modelo não linear para o ajuste de curvas deve considerar o comportamento dos dados experimentais e as restrições matemáticas da função. Alguns critérios importantes são:

1. **Análise gráfica:** representar os dados em diferentes escalas (linear, semilogarítmica ou logarítmica) para verificar se alguma transformação produz aproximadamente uma relação linear.

2. **Modelo exponencial:** para $y = be^{mx}$ ou $y = b10^{mx}$, com $b \neq 0$, a função não passa pela origem. A linearização logarítmica exige $y > 0$ (ou $y < 0$, quando se utiliza $\ln|y|$ e $b < 0$).

3. **Modelo logarítmico:** para $y = a + b\ln(x)$, o domínio exige $x > 0$.

4. **Modelo de potência:** para $y = bx^m$, com $m > 0$, tem-se $y = 0$ quando $x = 0$. A linearização por logaritmos exige $x > 0$ e $y > 0$, considerando $b > 0$.

5. **Modelo inverso:** para $y = \dfrac{1}{ax + b}$, exige-se $ax + b \neq 0$. Além disso, a transformação $Y = 1/y$ requer $y \neq 0$.

6. **Adequação do modelo:** verificar se a função representa o fenômeno físico estudado e se os parâmetros estimados são coerentes com o problema de Engenharia.

MN Aplicados à Engenharia

Exercícios de Fixação – Linearização com Python (I)

1. **Ajuste de uma função de potência** Considere os dados experimentais:

$$x = [1, 2, 3, 4, 5], \qquad y = [2.1, 8.3, 18.5, 32.2, 50.4].$$

Implemente um programa em Python que:

- transforme os dados usando $X = \ln(x)$ e $Y = \ln(y)$;
- determine os coeficientes a_1 e a_0 por mínimos quadrados;
- calcule os parâmetros $m = a_1$ e $b = e^{a_0}$;
- apresente o modelo ajustado $y = bx^m$;
- construa os gráficos dos dados experimentais e do modelo ajustado.

2. **Ajuste de uma função exponencial**
Considere os dados:

$$x = [0, 1, 2, 3, 4], \qquad y = [2.0, 3.3, 5.5, 9.0, 14.8].$$

Desenvolva um programa que:

- linearize o modelo $y = be^{mx}$;
- estime m e b por regressão linear;
- reconstrua a equação exponencial;
- compare os dados medidos com os valores previstos;
- apresente o gráfico de $\ln(y)$ em função de x.

MN Aplicados à Engenharia

Exercícios de Fixação – Linearização com Python (II)

3. **Linearização de uma exponencial de base 10** Considere os dados:

$$x = [0, 1, 2, 3, 4], \qquad y = [3.0, 6.0, 12.0, 24.0, 48.0].$$

Implemente um programa para:

- linearizar $y = b10^{mx}$ usando $\log_{10}(y)$;
- estimar os parâmetros m e b;
- comparar o modelo ajustado com os dados experimentais;
- verificar a relação entre o coeficiente angular e a taxa de crescimento exponencial.

4. **Comparação entre modelos não lineares** Considere os modelos

$$y = \frac{1}{mx + b}, \qquad y = \frac{mx}{b + x}.$$

Escolha ou gere conjuntos de dados compatíveis com cada modelo e implemente um programa que:

- aplique as transformações lineares apropriadas;
- estime os parâmetros por mínimos quadrados;
- reconstrua os modelos originais;
- compare os valores previstos com os dados;
- discuta as restrições e limitações de cada transformação.

5. **Análise dos resultados**
Para os modelos implementados, calcule o erro quadrático médio (EQM) na escala original e na escala transformada. Compare os resultados e explique por que minimizar o erro na escala transformada não garante o menor erro na escala original.

Organização

1. Ajustes de Curvas
   - Conceitos
2. Regressão Linear - mínimos quadrados
   - Implementação computacional
3. Análise de erros
   - Métricas de análise
4. Linearização
   - Conceitos
   - Implementação computacional
5. Regressão Polinomial
   - Implementação computacional
   - Implementação computacional
6. Polinômio interpolador de Lagrange
   - Implementação computacional
7. Polinômio interpolador de Newton
   - Implementação computacional
8. Splines: introdução
   - Splines Linear
     - Implementação computacional
   - Splines Quadráticas
     - Implementação computacional
   - Splines Cúbicas
     - Implementação computacional

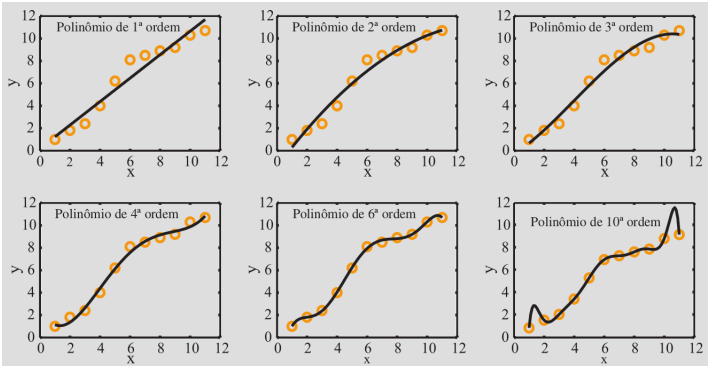
MN aplicados à Engenharia

Ajuste de curvas com polinômios de ordem superior

Considere a Figura 4. Note que:

1. Um polinômio de 1ª ordem representa uma reta; um de 2ª ordem, uma parábola; e um de 3ª ordem pode apresentar um ponto de inflexão, dependendo de seus coeficientes;
2. Quanto maior a ordem do polinômio, maior sua flexibilidade para representar variações e curvaturas nos dados;
3. A escolha do polinômio que proporciona o *melhor ajuste* depende da natureza e da origem dos dados, da aplicação e do objetivo da modelagem;
4. Observe as discrepâncias entre os dados observados e as curvas ajustadas nas regiões centrais e nas extremidades de cada gráfico da figura.

Figura 4: Polinômio de grau 'n'



Fonte: GILAT,(2008)

MN aplicados à Engenharia

Ajuste de curvas com polinômios quadráticos e de ordem superior

Considerações

1. É possível construir um polinômio interpolador que passe exatamente por n pontos distintos, desde que seu grau seja, no máximo, $(n-1)$;
2. Quando os dados apresentam incertezas ou imprecisões, o uso de polinômios de ordem elevada pode ser desnecessário e até inadequado;
3. Embora um polinômio interpolador de grau elevado reproduza exatamente os pontos utilizados na construção do modelo, ele pode apresentar oscilações significativas entre esses pontos;
4. Nessas condições, o polinômio pode não representar adequadamente o comportamento dos dados, prejudicando a confiabilidade das estimativas por interpolação e, principalmente, por extrapolação.

MN aplicados à Engenharia

Regressão polinomial

- Considere um polinômio de grau m, definido por:

$$f(x) = a_m x^m + a_{m-1} x^{m-1} + \cdots + a_1 x + a_0.$$

- Para um conjunto de n pontos observados (x_i, y_i), o ajuste polinomial pode ser formulado pela minimização da soma dos quadrados dos resíduos:

$$E = \sum_{i=1}^{n} \left[y_i - \left(a_m x_i^m + a_{m-1} x_i^{m-1} + \cdots + a_1 x_i + a_0 \right) \right]^2.$$

- Ao calcular as derivadas parciais de E em relação aos coeficientes $a_0, a_1, \ldots, a_m$ e igualá-las a zero, obtém-se um sistema de $m+1$ equações normais, cuja solução determina os coeficientes do polinômio ajustado.

- Para n pontos, quando $m = n-1$ e as abscissas são distintas, o polinômio interpolador pode reproduzir exatamente todos os pontos. Para $m < n-1$, o ajuste por mínimos quadrados busca minimizar o erro global, sem necessariamente passar por todos os pontos.

MN Aplicados à Engenharia

Considere um polinômio de **grau** $m = 2$, definido por:

$$f(x) = a_2x^2 + a_1x + a_0.$$

O ajuste por mínimos quadrados consiste em minimizar a soma dos quadrados dos resíduos:

$$E = \sum_{i=1}^{n} \Big[y_i - (a_2x_i^2 + a_1x_i + a_0) \Big]^2.$$

- Para determinar os coeficientes a_0, a_1 e a_2, calculam-se as derivadas parciais de E em relação a cada coeficiente e igualam-se a zero.
- As condições de mínimo são expressas por:

$$\frac{\partial E}{\partial a_0} = -2\sum_{i=1}^{n} \Big[y_i - (a_2x_i^2 + a_1x_i + a_0) \Big] = 0,$$

$$\frac{\partial E}{\partial a_1} = -2\sum_{i=1}^{n} x_i \Big[y_i - (a_2x_i^2 + a_1x_i + a_0) \Big] = 0,$$

$$\frac{\partial E}{\partial a_2} = -2\sum_{i=1}^{n} x_i^2 \Big[y_i - (a_2x_i^2 + a_1x_i + a_0) \Big] = 0.$$

MN Aplicados à Engenharia

Ajuste de Curvas e Interpolação

A partir das condições de minimização do erro, obtém-se o seguinte **sistema de equações normais**:

$$\begin{cases} na_0 + \left(\sum_{i=1}^{n} x_i\right) a_1 + \left(\sum_{i=1}^{n} x_i^2\right) a_2 = \sum_{i=1}^{n} y_i, \\ \left(\sum_{i=1}^{n} x_i\right) a_0 + \left(\sum_{i=1}^{n} x_i^2\right) a_1 + \left(\sum_{i=1}^{n} x_i^3\right) a_2 = \sum_{i=1}^{n} x_i y_i, \\ \left(\sum_{i=1}^{n} x_i^2\right) a_0 + \left(\sum_{i=1}^{n} x_i^3\right) a_1 + \left(\sum_{i=1}^{n} x_i^4\right) a_2 = \sum_{i=1}^{n} x_i^2 y_i. \end{cases}$$

- A solução desse sistema determina os coeficientes a_0, a_1 e a_2 do polinômio quadrático

$$f(x) = a_2 x^2 + a_1 x + a_0,$$

que minimiza a soma dos quadrados dos resíduos associados aos n pontos observados (x_i, y_i).

- O mesmo procedimento pode ser generalizado para polinômios de graus superiores, resultando em um sistema de equações normais cujas dimensões dependem do grau do polinômio.

MN Aplicados à Engenharia

Formulação Matricial do Ajuste Polinomial

As equações normais podem ser organizadas na forma matricial:

$$\underbrace{\begin{bmatrix} n & \sum x_i & \sum x_i^2 \\ \sum x_i & \sum x_i^2 & \sum x_i^3 \\ \sum x_i^2 & \sum x_i^3 & \sum x_i^4 \end{bmatrix}}_{A} \underbrace{\begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix}}_{\boldsymbol{a}} = \underbrace{\begin{bmatrix} \sum y_i \\ \sum x_i y_i \\ \sum x_i^2 y_i \end{bmatrix}}_{\boldsymbol{b}}, \qquad i = 1, \ldots, n.$$

- O sistema assume a forma compacta

$$A\boldsymbol{a} = \boldsymbol{b}.$$

  Se A for não singular, sua solução pode ser expressa formalmente por $\boldsymbol{a} = A^{-1}\boldsymbol{b}$.

- Para fins de implementação computacional, observe:
  - Os elementos de A seguem um padrão baseado nas somas das potências de x_i; os elementos de $\boldsymbol{b}$ envolvem os produtos entre y_i e as potências de x_i.
  - O vetor de coeficientes é organizado em ordem crescente de grau: $\boldsymbol{a} = [a_0, a_1, a_2]^T$.
  - Essa convenção corresponde à representação

$$p(x) = a_0 + a_1 x + \cdots + a_m x^m.$$

  Caso se utilize a ordem decrescente, $[a_m, \ldots, a_1, a_0]$, a implementação deverá respeitar essa convenção.

MN Aplicados à Engenharia

Implementação Computacional: Regressão Polinomial

Código Python

```python
import numpy as np                                                    # Operacoes numericas
import matplotlib.pyplot as plt                                       # Graficos
x  = np.arange(1, 9)                                                  # Valores de entrada
y  = np.array([2.1, 3.9, 4.2, 6.1, 7.5, 8.3, 8.9, 10.4])              # Dados observados
m  = 2                                                                # Grau do polinomio
a  = np.polyfit(x, y, m)                                              # Estima os coeficientes
xx = np.linspace(x.min(), x.max(), 200)                               # Pontos para a curva
yy = np.polyval(xx, a)                                                # Avalia o polinomio
print("Coeficientes:", a)                                             # Exibe os coeficientes
plt.scatter(x, y, label="Dados observados")                           # Plota os dados
plt.plot(xx, yy, "r-", label="Regressao polinomial")                  # Plota o ajuste
plt.xlabel("x")                                                       # Rotulo do eixo x
plt.ylabel("y")                                                       # Rotulo do eixo y
plt.legend()                                                          # Exibe a legenda
plt.grid(True)                                                        # Ativa a grade
plt.show()                                                            # Exibe o grafico
```

MN Aplicados à Engenharia

Exercício Computacional: Regressão Polinomial

Considere os dados experimentais abaixo e utilize Python para realizar o ajuste polinomial pelo método dos mínimos quadrados.

Dados experimentais

$$x = [1, 2, 3, 4, 5, 6, 7, 8]$$

$$y = [2.1, 3.9, 4.2, 6.1, 7.5, 8.3, 8.9, 10.4]$$

1. **Implementação:** escreva um programa em Python para ajustar um polinômio de grau $m = 2$ aos dados experimentais, utilizando a função `polyfit` da biblioteca NumPy.
2. **Análise dos coeficientes:** apresente os coeficientes calculados pelo programa e escreva a equação do polinômio ajustado na forma
$$p(x) = a_0 + a_1 x + a_2 x^2.$$
3. **Visualização e comparação:** construa um gráfico contendo os pontos experimentais e a curva ajustada. Repita o procedimento para os graus $m = 1, 2, 3, 4$ e $m = 5$ e compare os resultados, identificando as diferenças entre os ajustes. Apresente um tabela com resultados comparativos.

MN Aplicados à Engenharia

Exercício Computacional: Avaliação da Regressão Polinomial

Considere os dados experimentais abaixo e avalie a qualidade dos modelos polinomiais obtidos pelo método dos mínimos quadrados.

Dados experimentais ($n = 20$)

$$x = [1.2, 1.8, 2.5, 3.1, 3.7, 4.2, 4.9, 5.4, 6.0, 6.6, 7.1, 7.8, 8.3, 8.9, 9.5, 10.2, 10.8, 11.4, 12.1, 12.8]$$

$$y = [3.4, 4.1, 4.8, 5.2, 6.0, 6.3, 7.1, 7.4, 8.2, 8.5, 9.1, 9.8, 10.0, 10.7, 11.1, 12.0, 12.3, 13.1, 13.5, 14.2]$$

1. **Previsão:** ajuste um polinômio de grau $m = 2$ e estime y para $x = 7.0$, utilizando a função `polyval` da biblioteca NumPy.
2. **Análise dos resíduos:** calcule $e_i = y_i - p(x_i)$ e construa um gráfico dos resíduos em função de x. Verifique se os resíduos apresentam padrões sistemáticos.
3. **Comparação dos modelos:** calcule a soma dos quadrados dos resíduos (SSE) para os polinômios de graus 1 ao 10. Compare os resultados e discuta os efeitos do aumento da complexidade do modelo. Apresente os resultados numa tabela.

Organização

1. Ajustes de Curvas
   - Conceitos
2. Regressão Linear - mínimos quadrados
   - Implementação computacional
3. Análise de erros
   - Métricas de análise
4. Linearização
   - Conceitos
   - Implementação computacional
5. Regressão Polinomial
   - Implementação computacional
   - Implementação computacional
6. Polinômio interpolador de Lagrange
   - Implementação computacional
7. Polinômio interpolador de Newton
   - Implementação computacional
8. Splines: introdução
   - Splines Linear
     - Implementação computacional
   - Splines Quadráticas
     - Implementação computacional
   - Splines Cúbicas
     - Implementação computacional

MN aplicados à Engenharia

Ajuste de Curvas e interpolação

Polinômio de Lagrange

- A solução de um ajuste de curvas envolve com polinômio **padrão**. Procedimento:
  1. Definir o polinômio cujo formato padrão é: $f(x) = a_m x^m + a_{m-1} x^{m-1} + \ldots + a_1 x + a_0$;
  2. Estruturar o sistema na forma matricial ($Ax = B$), para obter $x = A^{-1}B$;
  3. Verificar se o grau do polinômio encontrado converge nos pontos de análise e região de vizinhança sem *saltos* entre pontos;

  Nota: Ineficiente para polinômios de **ordem mais elevada** e quando a matriz dos coeficientes é mal condicionada.
- Os *polinômios interpoladores de Lagrange*
  1. Formam uma classe específica de polinômios que podem ser utilizados para fazer o ajuste de um determinado conjunto de dados simplesmente a partir dos valores dos pontos;
  2. Os polinômios podem ser escritos diretamente e os coeficientes são determinados sem a necessidade de nenhum cálculo preliminar.

MN aplicados à Engenharia

Polinômio de Lagrange

- Para dois pontos $a = (x_1, y_1)$ e $b = (x_2, y_2)$, o polinômio de Lagrange tem a seguinte forma:
  - $f(x) = y = a_1(x - x_2) + a_2(x - x_1)$, o qual para os pontos a e b resultado em:
  * $y_1 = a_1(x_1 - x_2) + a_2(x_1 - x_1) \rightarrow a_1 = \dfrac{y_1}{(x_1 - x_2)}$
  * $y_2 = a_1(x_2 - x_2) + a_2(x_2 - x_1) \rightarrow a_2 = \dfrac{y_2}{(x_2 - x_1)}$

  Recompondo a equação, temos:
  - $f(x) = \dfrac{(x - x_2)}{(x_1 - x_2)} y_1 + \dfrac{(x - x_1)}{(x_2 - x_1)} y_2$

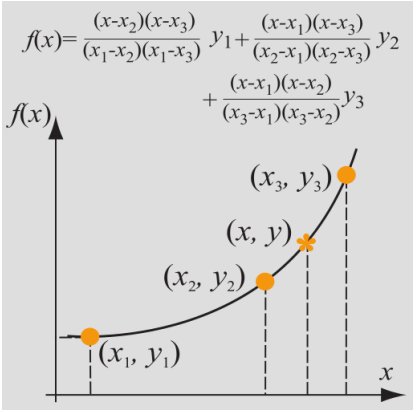
MN aplicados à Engenharia

Polinômio de Lagrange

- Para três pontos $(x_1, y_1), (x_2, y_2)$ e (x_3, y_3) o polinômio de Lagrange tem a seguinte forma:

$$f(x) = \frac{(x-x_2)(x-x_3)}{(x_1-x_2)(x_1-x_3)}y_1 + \frac{(x-x_1)(x-x_3)}{(x_2-x_1)(x_2-x_3)}y_2 + \frac{(x-x_1)(x-x_2)}{(x_3-x_1)(x_3-x_2)}y_3$$

Figura 5: Lagrange de 2ª ordem



Fonte: GILAT,(2008)

MN aplicados à Engenharia

Ajuste de Curvas e interpolação

Polinômio de Lagrange

- Escrito de forma compacta utilizando-se a notação de soma e produto o polinômio fica:

$$f(x) = \sum_{i=1}^{n} y_i L_i(x) = \sum_{i=1}^{n} y_i \prod_{j=1,\ j\neq i}^{n} \frac{(x - x_j)}{(x_i - x_j)},$$

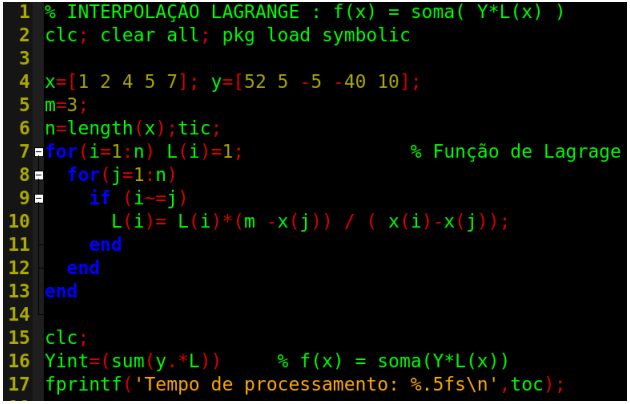
em que as funções $L_i(x) = \prod_{j=1,\ j\neq i}^{n} \frac{(x - x_j)}{(x_i - x_j)}$ são chamadas de funções de Lagrange.

- **Considerações**
  - Os pontos não precisam estar equidistantes em si;
  - Deve-se calcular a expressão completa do polinômio interpolador para cada valor de x a ser interpolado;
  - Uma vez ampliado o conjunto de dados, todos os termos do polinômio de Lagrange devem ser calculados novamente.

MN aplicados à Engenharia

Ajuste de Curvas e interpolação - Exemplo

Figura 6: Exemplo de *script* de interpolação por Lagrange no Octave/Matlab



```
% INTERPOLAÇÃO LAGRANGE : f(x) = soma( Y*L(x) )
clc; clear all; pkg load symbolic

x=[1 2 4 5 7]; y=[52 5 -5 -40 10];
m=3;
n=length(x);tic;
for(i=1:n) L(i)=1;                  % Função de Lagrage
  for(j=1:n)
    if (i~=j)
      L(i)= L(i)*(m -x(j)) / ( x(i)-x(j));
    end
  end
end

clc;
Yint=(sum(y.*L))      % f(x) = soma(Y*L(x))
fprintf('Tempo de processamento: %.5fs\n',toc);
```

Fonte: Autor,(2020)

MN aplicados à Engenharia

Ajuste de Curvas e interpolação - Exemplo

Figura 7: Exemplo de *script* de interpolação por Lagrange no Python

```python
import numpy as np
# ================================================================================
x=np.array([1,2,4,5,7])
y=np.array([52,5,-5,-40,10])
#%% OPÇÃO 01=====================================================================
p=3
k=np.ones(len(x))                          # vetor de uns para 1° elemento
for i in range(len(x)):
    for j in range(len(x)):
        if (i!=j):
            k[i]=k[i]*(p-x[j])/(x[i]-x[j])
Yint=sum(y*k)
print(Yint)
```

Fonte: Autor,(2024)

Organização

1. Ajustes de Curvas
   - Conceitos
2. Regressão Linear - mínimos quadrados
   - Implementação computacional
3. Análise de erros
   - Métricas de análise
4. Linearização
   - Conceitos
   - Implementação computacional
5. Regressão Polinomial
   - Implementação computacional
   - Implementação computacional
6. Polinômio interpolador de Lagrange
   - Implementação computacional
7. Polinômio interpolador de Newton
   - Implementação computacional
8. Splines: introdução
   - Splines Linear
     - Implementação computacional
   - Splines Quadráticas
     - Implementação computacional
   - Splines Cúbicas
     - Implementação computacional

MN aplicados à Engenharia

Ajuste de Curvas e interpolação

Polinômio de Newton

- Método alternativo ao polinômio interpolador de Lagrange
- Nos *polinômios interpoladores de Newton*
  - A determinação dos coeficientes não requer a solução de um sistema com n equações;
  - Os pontos do conjunto de dados não precisam estar ordenados de forma ascendente ou descendente;
  - Os coeficientes a_1 a a_n, uma vez determinados, podem ser utilizados para interpolar quaisquer dos pontos que compõem o conjunto de dados;
  - Após a determinação dos n coeficientes de um polinômio interpolador de Newton de ordem $n-1$, mais pontos podem ser adicionados ao conjunto de dados, sendo necessário apenas determinar os coeficientes adicionais;
  - A forma geral do polinômio de Newton de ordem $n-1$ que passa por n pontos é:
    $f(x) = a_1 + a_2(x - x_1) + a_3(x - x_1)(x - x_2) + \ldots + a_n(x - x_1)(x - x_2)\ldots(x - x_{n-1})$

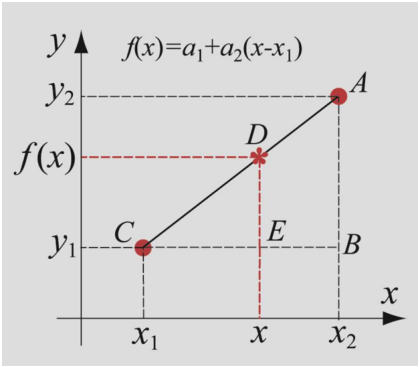
MN aplicados à Engenharia

Ajuste de Curvas e interpolação

Polinômio de Newton de primeira ordem

$$f(x) = a_1 + a_2(x - x_1) + a_3(x - x_1)(x - x_2) + \ldots + a_n(x - x_1)(x - x_2)\ldots(x - x_{n-1})$$

Figura 8: Polinômio de Newton



Fonte: GILAT (2008)

Para dois pontos (x_1, y_1) e (x_2, y_2) o polinômio de Newton tem a seguinte forma:

$$f(x) = a_1 + a_2(x - x_1)$$

Os coeficientes a_1 e a_2 podem ser calculados utilizando-se semelhança de triângulos, conforme figura ao lado.
Desse modo: $f(x) = y1 + \frac{y_2 - y_1}{x_2 - x_1}(x - x_1)$, portanto, $a_1 = y_1$ e $a_2 = \dfrac{y_2 - y_1}{x_2 - x_1}$.

MN aplicados à Engenharia

Ajuste de Curvas e interpolação

Polinômio de Newton de ordem 2 e 3

- Para três pontos $(x_1, y_1), (x_2, y_2)$ e (x_3, y_3) o polinômio de Newton tem a seguinte forma:

$$f(x) = a_1 + a_2(x - x_1) + a_3(x - x_1)(x - x_2)$$

- Os valores de a_1 e a_2 são os mesmos obtidos para dois pontos. $a_1 = y_1$ e $a_2 = \dfrac{y_2 - y_1}{x_2 - x_1}$
  Através de manipulações matemáticas, tem-se:
- O termo adicional para um polinômio de ordem 2 seria: $a_3 = \dfrac{\frac{y_3 - y_2}{x_3 - x_2} - \frac{y_2 - y_1}{x_2 - x_1}}{x_3 - x_1}$
- O termo adicional para um polinômio de ordem 3 seria: $a_4 = \dfrac{\frac{y_3 - y_2}{x_3 - x_2} - \frac{y_2 - y_1}{x_2 - x_1}}{x_4 - x_1}$

MN aplicados à Engenharia

Polinômio de Newton - Ordem superior

Para encontrar os coeficientes maiores que a_3, utiliza-se o conceito das **diferenças divididas ($\Delta x/\Delta y$)**.

- $f[x_2, x_1] = \frac{y_2 - y_1}{x_2 - x_1} = a_2$
- $f[x_3, x_2, x_1] = \frac{f[x_3, x_2] - f[x_2, x_1]}{x_3 - x_1} = \frac{\left(\frac{y_3 - y_2}{x_3 - x_2}\right) - \left(\frac{y_2 - y_1}{x_2 - x_1}\right)}{x_3 - x_1} = a_3$
- $f[x_4, x_3, x_2, x_1] = \frac{f[x_4, x_3, x_2] - f[x_3, x_2, x_1]}{x_4 - x_1} = \frac{\frac{f[x_4, x_3] - f[x3, x_2]}{x_4 - x_2} - \frac{f[x_3, x_2] - f[x_2, x_1]}{x_3 - x_1}}{x_4 - x_1} = a_4 \ldots$
- $f[x_5, x_4, x_3, x_2, x_1] = \frac{f[x_5, x_4, x_3, x_2] - f[x_4, x_3, x_2, x_1]}{x_5 - x_1} = a_5 \ldots$
- $f[x_6, x_5, x_4, x_3, x_2, x_1] = \frac{f[x_6, x_5, x_4, x_3, x_2] - f[x_5, x_4, x_3, x_2, x_1]}{x_6 - x_1} = a_6 \ldots$

MN aplicados à Engenharia

Ajuste de Curvas e interpolação

Polinômio de Newton - Forma geral

Portanto, a k-ésima *diferença dividida*, ordem 2 ou superior, é dada por:

- $f[x_k, x_{k-1}, x_2, x_1] = \dfrac{f[x_k, x_{k-1}, x_3, x_2] - f[x_{k-1}, x_{k-2}, \ldots, x_2, x_1]}{x_k - x_1}$,
  em que a ordem máxima é $(n-1)$
- **Forma geral**:
  $f(x) = \begin{cases} y_1 + f[x_2, x_1](x - x_1) + f[x_3, x_2, x_1](x - x_1)(x - x_2) + \ldots \\ \quad \cdots + f[x_n, x_{n-1}, x_2, x_1](x - x_1)(x - x_2)\ldots(x - x_{n-1}) \end{cases}$

MN aplicados à Engenharia

Figura 9: Exemplo de *script* de interpolação por Newton no Octave/Matlab

```
% INTERPOLAÇAO NEWTON :
clc; clear all;  pkg load symbolic

x=[1 2 4 5 7]; y=[52 5 -5 -40 10];
xint=3;
n=length(x);
a(1)=y(1);
for (i=1:n-1)
  divDIF(i,1)=(y(i+1)-y(i))/(x(i+1)-x(i));     %Diferenças na coluna 1
end
for(j=2:n-1)
  for(i=1:n-j)
  divDIF(i,j)=(divDIF(i+1,j-1)-divDIF(i,j-1))/(x(j+i)-x(i));
  end
end
for(j=2:n) a(j)=divDIF(1,j-1);end
Yint=a(1); xn=1;
for(k=2:n) xn=xn*(xint -x(k-1)); Yint=Yint+a(k)*xn; end
Yint
```

Fonte: Adaptado de GILAT,(2008)

MN aplicados à Engenharia

Figura 10: Exemplo de *script* de interpolação por Newton em Python

```python
import numpy as np
x=np.array([1,2,4,5,7])
y=np.array([52,5,-5,-40,10])
print('\014')
xint=3
a,s= [y[0]],y                                  # s é vetor das divisoes
for i in range(len(x)-1):
    y=s                                        # Atualiza o vetor y
    s=(y[1:]-y[:-1])/(x[(1+i):]-x[:-(1+i)])    # Reduz o vetor x
    a.append(s[0])
xn,Yint=1,a[0]
for k in range(1,len(x)):
    xn=xn*(xint - x[k-1])
    Yint=Yint+a[k]*xn
print('\nA aproximação encontrada para f(%.1f) = %.2f'%(xint,Yint))
```

Fonte: Auttor,(2024)

Organização

1. Ajustes de Curvas
   - Conceitos
2. Regressão Linear - mínimos quadrados
   - Implementação computacional
3. Análise de erros
   - Métricas de análise
4. Linearização
   - Conceitos
   - Implementação computacional
5. Regressão Polinomial
   - Implementação computacional
   - Implementação computacional
6. Polinômio interpolador de Lagrange
   - Implementação computacional
7. Polinômio interpolador de Newton
   - Implementação computacional
8. Splines: introdução
   - Splines Linear
     - Implementação computacional
   - Splines Quadráticas
     - Implementação computacional
   - Splines Cúbicas
     - Implementação computacional

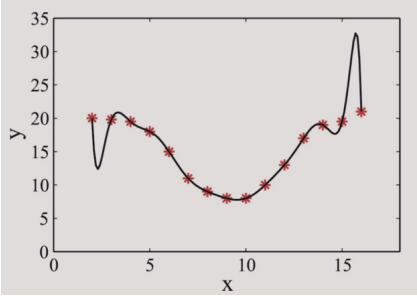
MN aplicados à Engenharia

Ajuste de Curvas e interpolação

Splines: introdução

- Quando um conjunto de dados contendo *n* pontos é dado e um *único polinômio* é utilizado para fazer a sua interpolação, esse polinômio fornece os valores exatos nos pontos e determina valores estimados (interpolados) entre eles;
- Entretanto, quanto maior o valor de *n*, a natureza oscilatória dos polinômios pode causar fenômenos indesejáveis, conforme figura a seguir.

Figura 11: Spline



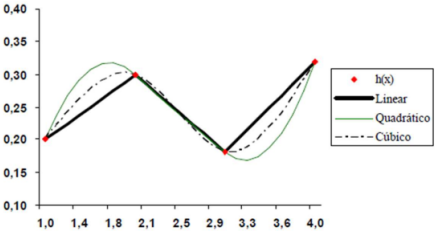
MN aplicados à Engenharia

Ajuste de Curvas e interpolação

Splines - Introdução

- Quando se trabalha com um grande número de pontos, uma melhor interpolação pode ser feita com o uso de *muitos polinômios de baixa ordem* ao invés de um único polinômio de ordem elevada;
- Cada polinômio de baixa ordem é válido em um intervalo entre dois ou vários pontos;
- Tipicamente, todos os polinômios utilizados têm a mesma ordem, mas os coeficientes são diferentes em cada intervalo;
- A interpolação feita dessa forma é chamada de interpolação por partes, ou *spline*.
- Os pontos do conjunto de dados em que se encontram os polinômios de intervalos adjacentes são chamados de nós.

Figura 12: Spline em interpolação



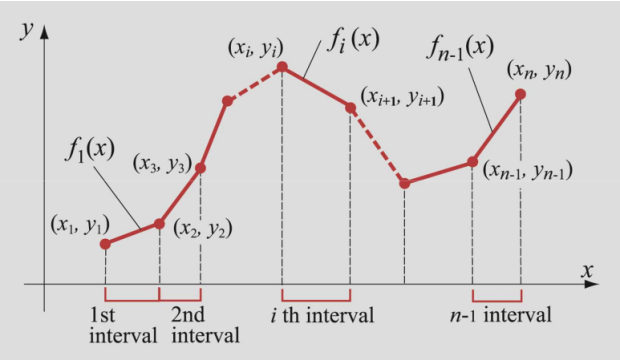
Fonte: GILAT,(2008)

- Os três tipos de interpolação *spline*:
  - Linear
  - Quadrática
  - Cúbica

MN aplicados à Engenharia

Splines Linear

Figura 13: Spline Linear



Fonte: GILAT,(2008)

MN aplicados à Engenharia

Ajuste de Curvas e interpolação

Splines Linear

- Numa *spline* linear, a interpolação é feita utilizando-se um polinômio de primeira ordem, e os pontos são conectados por linhas retas;
- Utilizando-se o polinômio interpolador de Lagrange, a equação da linha reta que conecta os dois primeiros pontos é dada por:

$$p(x) = s_1(x) = \frac{(x - x_2)}{(x_1 - x_2)} y_1 + \frac{(x - x_1)}{(x_2 - x_1)} y_2$$

- A interpolação no intervalo i, que está entre os pontos x_i e x_{i+1} ($x_i \leq x \leq x_{i+1}$), é feita pela equação da linha reta que conecta o ponto (x_i, y_i) ao ponto (x_{i+1}, y_{i+1}):

$$p(x) = s_i(x) = \frac{(x - x_{i+1})}{(x_i - x_{i+1})} y_i + \frac{(x - x_i)}{(x_{i+1} - x_i)} y_{i+1},$$

para $i = 1, 2, \ldots, n - 1$

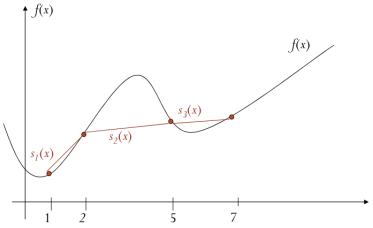
MN aplicados à Engenharia

Ajuste de Curvas e interpolação

Splines Linear

- Note que $s_i(x)$ é um polinômio de grau 1 no intervalo;
- $s_1(x)$ é **contínua** em todo intervalo $[x_{i-1}, x_i]$, já que os dois polinômios adjacentes têm o mesmo valor em um nó comum;
- Nos nós $s_i(x_i) = f(x_i)$;
- Entretanto, uma descontinuidade na inclinação das *splines* lineares nos nós está presente.

Figura 14: Spline Linear



Fonte: DIAS,(2019)

MN aplicados à Engenharia

Exemplo: Determine as *splines* lineares que fazem o ajuste dos dados: $x = [8, 11, 15, 18, 22]; y = [5, 9, 10, 8, 7]$.

- **Os cinco pontos geram quatro splines, portanto:**
  - Para $8 \leq x \leq 11$
    $$f_1(x) = \frac{(x - x_2)}{(x_1 - x_2)} y_1 + \frac{(x - x_1)}{(x_2 - x_1)} y_2 = \frac{(x - 11)}{(8 - 11)} 5 + \frac{(x - 8)}{(11 - 8)} 9 = \frac{5}{-3}(x - 11) + \frac{9}{2}(x - 8)$$
  - Para $11 \leq x \leq 15$
    $$f_2(x) = \frac{(x - x_3)}{(x_2 - x_3)} y_2 + \frac{(x - x_2)}{(x_3 - x_2)} y_3 = \frac{(x - 15)}{(11 - 15)} 9 + \frac{(x - 11)}{(15 - 11)} 10 = \frac{9}{-4}(x - 15) + \frac{10}{4}(x - 11)$$
  - Para $15 \leq x \leq 18$
    $$f_3(x) = \frac{(x - x_4)}{(x_3 - x_4)} y_3 + \frac{(x - x_3)}{(x_4 - x_3)} y_4 = \frac{(x - 18)}{(15 - 18)} 10 + \frac{(x - 15)}{(18 - 15)} 8 = \frac{10}{-3}(x - 18) + \frac{8}{3}(x - 15)$$
  - Para $18 \leq x \leq 22$
    $$f_4(x) = \frac{(x - x_5)}{(x_4 - x_5)} y_4 + \frac{(x - x_4)}{(x_5 - x_4)} y_5 = \frac{(x - 22)}{(18 - 22)} 8 + \frac{(x - 18)}{(22 - 18)} 7 = -2(x - 18) + \frac{7}{4}(x - 22)$$

- O **valor interpolado de y em $x = 12,7$ é obtido com a substituição do valor x na equação de $f_2(x)$ acima resultando em:**
  $$f_2(x) = \frac{9}{-4}(x - 15) + \frac{10}{4}(x - 11) = \frac{9}{-4}(12,7 - 15) + \frac{10}{4}(12,75 - 11) = \mathbf{9,425}$$

MN aplicados à Engenharia

Ajuste de Curvas e interpolação

Figura 15: *Script* Spline Linear no Octave/Matlab

```
% Interpolaçao Parcial Linear - SPLINE LINEAR
clc; clear all;  pkg load symbolic

%************************************************************
x=[8 11 15 18 22]; y=[5 9 10 8 7]; x_int=12.7;

%************************************************************
n=length(x);                    % Encontrando o intervalo de interpolacao
for(i=1:n)
      if x_int < x(1) | x_int > x(n)
      error('Interpolação fora do intervalo')
      elseif x_int<x(i+1) break  % Fim da busca pelo intervalo
      end
 end
%************************************************************
syms m x1 x2 y1 y2
clc;
px=simplify((m-x2)*y1/(x1-x2) + (m-x1)*y2/(x2-x1));     % Polinômio
f=matlabFunction(px);                          % Função com 5 termos
y_int = f( x_int, x(i), x(i+1), y(i), y(i+1));  % Dados de intrada para i
pol=subs(px,'m',x_int)
fprintf('Intervalo de interpolação para x= %d é %d < x < %d.\n\n',
x_int,x(i),x(i+1));
fprintf('O ponto f(%.2f) = %.2f \n\n',x_int,y_int);

%************************************************************
plot(x_int,y_int,'*'); hold on; plot(x,y,'0');
legend('Pontos x e y');
```

Fonte: AUTOR (2026)

MN aplicados à Engenharia

Ajuste de Curvas e interpolação

Figura 16: *Script* Spline Linear em Pyhton

```
import matplotlib.pyplot as plt
# ========================================================================
x=[8,11,15,18]
y=[5,9,10,8]
xint=12.7
# ========================================================================
print('\014')
# n=len(x)
for i in range(len(x)):
    if (xint < x[0] and xint >x[-1]):
        print('Erro!\nInterpolação fora do intervalo.')
    elif xint<x[i+1]:
        i+=1
        break
yint=(xint-x[i])* y[i-1]/ (x[i-1]-x[i]) + (xint-x[i-1])*y[i]/(x[i]-x[i-1])
print('\nA aproximação encontrada para f(%.1f) = %.2f'%(xint,yint))

plt.plot(x,y,'b',label='Pontos de medição')
plt.plot(xint,yint,'r*',label='y interpolado')
plt.legend()
plt.title('Interpolação via spline linear')
plt.style.use('ggplot')
```

Fonte: AUTOR,(2024)

MN aplicados à Engenharia

Ajuste de Curvas e interpolação

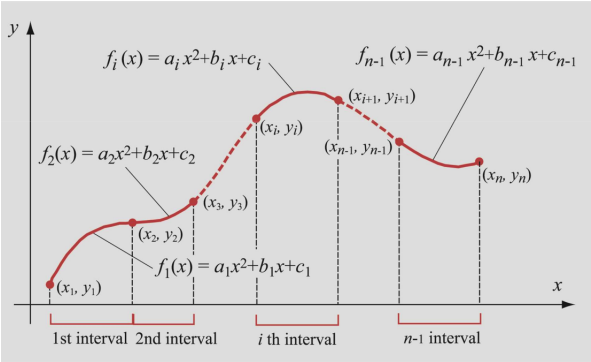
Splines Linear

- Uma desvantagem da interpolação linear é que a função de interpolação é geralmente não diferenciável nos pontos extremos de cada intervalo.
- Dessa forma, a função não é suave nos pontos.
- Para uma melhor aproximação de sistemas reais, a característica de suavidade é desejável, logo, a função de interpolação deve ser contínua e diferenciável.
- Proposições de melhor ajuste utilizam função de ordem superior.

MN aplicados à Engenharia

- Nas *splines* quadráticas, a interpolação é feita com polinômios de **segunda ordem**;
- Em um conjunto de n pontos, há $n-1$ intervalos, portanto, a equação no i-ésimo intervalo, localizado entre os pontos x_i e x_{i+1} é dada por $f_i(x) = a_i x^2 + b_i x + c_i$ para $i = 1, 2, \ldots, n-1$;

Figura 17: Splines Quadráticas



Fonte: GILAT,(2008)

MN aplicados à Engenharia

Ajuste de Curvas e interpolação

Splines Quadráticas

(1) De forma geral, há $n-1$ equações e, como cada equação tem três coeficientes, um total de $3(n-1) = 3n-3$ coeficientes precisam determinados de tal forma que:

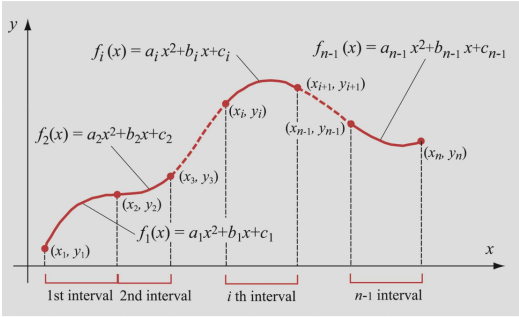
* Cada polinômio $f_i(x)$ deve passar pelos pontos finais do intervalo, (x_i, y_i) e (x_{i+1}, y_{i+1}), o que significa que $f_i(x_i) = y_i$ e $f_i(x_{i+1}) = y_{i+1}$, portanto:

$$a_i x_i^2 + b_i x_i + c_i = y_i$$

$$a_i x_{i+1}^2 + b_i x_{i+1} + c_i = y_{i+1}$$

para $i = 1, 2, \ldots, n-1$.

Figura 18: Splines Quadráticas



Fonte: GILAT,(2008)

MN aplicados à Engenharia

Ajuste de Curvas e interpolação

Splines Quadráticas

(2) Nos nós internos, as inclinações (derivadas primeiras) dos polinômios de intervalos adjacentes são iguais. Isso significa que, com a transição da curva que passa por um nó interno de um polinômio para outro, a inclinação deve ser contínua.
A derivada primeira do i-ésimo polinômio é: $f'(x) = \frac{df}{dx} = 2a_i x + b_i$

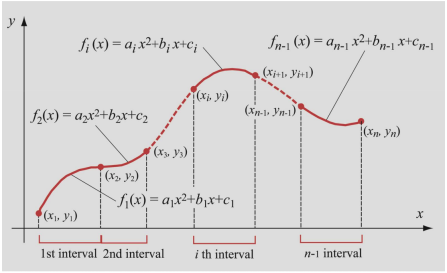
* Para n pontos, o primeiro ponto interno é $i = 2$ e o último é $i = n - 1$. Igualando-se as derivadas primeiras em todos os **pontos internos**, obtém-se:

$$2a_{i-1}x_i + b_{i-1} = 2a_i x_i + b_i$$

para $i = 2, 3, \ldots, n - 1$.

* Como há $n-2$ pontos internos, essa condição fornece $n-2$ equações, portanto, juntas às equações do passo (1) tem-se um total de $3n - 4$ equações.

Figura 19: Splines Quadráticas



Fonte: GILAT,(2008)

MN aplicados à Engenharia

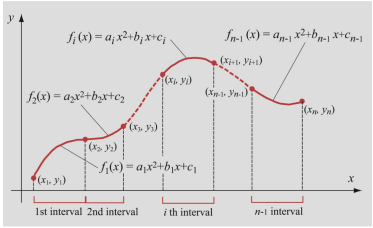
Ajuste de Curvas e interpolação

Splines Quadráticas

(3) Entretanto, os $n-1$ polinômios têm $3n-3$ coeficientes, de forma que uma equação adicional (condição) é necessária para que os coeficientes sejam obtidos. A condição comumente aplicada assume que a derivada segunda seja nula no **primeiro ou no último ponto**.

- A derivada **segunda** no primeiro ponto, (x_1, y_1), é nula. O polinômio no primeiro intervalo (entre o primeiro e o segundo ponto) é:
  - $f_1(x) = a_1x^2 + b_1x + c_1$
  - A derivada segunda do polinômio é $f_1''(x) = 2a_1$, que, quando igualada a zero, resulta em $a_1 = 0$
  - A interpretação dessa condição é que uma linha reta conecta os dois primeiros pontos.

Figura 20: Splines Quadráticas



Considerações

- *Splines* quadráticas têm derivada primeira contínua em pontos internos (nós);
- Em um conjunto de n pontos, elas requerem a solução de um sistema linear com $3n-4$ equações para que os coeficientes dos polinômios sejam determinados.

MN aplicados à Engenharia

Exemplo: Determine as *splines* quadráticas que fazem o ajuste dos dados: $x = [8, 11, 15, 18, 22]; y = [5, 9, 10, 8, 7]$ e encontre $f(12, 7)$.

- **Os cinco pontos geram quatro splines**. A equação quadrática para a i-ésima spline é: $f_i(x) = a_i x^2 + b_i x + c_i$
  - Há quatro polinômios, e, como cada polinômio tem três coeficientes, 12 coeficientes têm que ser determinados no total.
  - Os coeficientes são $a_1, b_1, c_1, a_2, b_2, c_2, a_3, b_3, c_3, a_4, b_4 e c_4$.
  - O coeficiente a_1 **é igual a zero** e os outros 11 coeficientes são determinados a partir de um sistema linear de 11 equações.
  - Oito equações são obtidas a partir da condição que diz que, em cada intervalo, o polinômio deve passar pelos pontos finais e, portanto:

$$
\begin{array}{ll}
\text{i=1} & f_1(x) = a_1 x_1^2 + b_1 x_1 + c_1 = (0).8^2 + b_1 8 + c_1 = 5 \\
& f_1(x) = a_1 x_2^2 + b_1 x_2 + c_1 = (0).11^2 + b_1 11 + c_1 = 9 \\
\text{i=2} & f_2(x) = a_2 x_2^2 + b_2 x_2 + c_2 = a_2 11^2 + b_2 11 + c_2 = 9 \\
& f_2(x) = a_2 x_3^2 + b_2 x_3 + c_2 = a_2 15^2 + b_2 15 + c_2 = 10 \\
\text{i=3} & f_3(x) = a_3 x_3^2 + b_3 x_3 + c_3 = a_3 15^2 + b_3 15 + c_3 = 10 \\
& f_3(x) = a_3 x_4^2 + b_3 x_4 + c_3 = a_3 18^2 + b_3 18 + c_3 = 8 \\
\text{i=4} & f_4(x) = a_4 x_4^2 + b_4 x_4 + c_4 = a_4 18^2 + b_4 18 + c_4 = 8 \\
& f_4(x) = a_4 x_5^2 + b_4 x_5 + c_4 = a_4 22^2 + b_4 22 + c_4 = 7
\end{array}
$$

MN aplicados à Engenharia

Splines Quadráticas - Exemplo

- Três equações são obtidas a partir da condição que diz que, nos nós interiores, as inclinações (derivadas primeiras) dos polinômios de intervalos adjacentes são iguais, portanto:

  i=2 $2a_1x_2 + b_1 = 2a_2x_2 + b_2 \rightarrow$
  $b_1 = 2a_211 + b_2$
  i=3 $2a_2x_3 + b_2 = 2a_3x_3 + b_3 \rightarrow$
  $2a_215 + b_2 = 2a_315 + b_3$
  i=4 $2a_3x_4 + b_3 = 2a_4x_4 + b_4 \rightarrow$
  $2a_318 + b_3 = 2a_418 + b_4$

- O sistema de 11 equações lineares pode ser escrito na forma matricial;
- Note, na figura, que os coeficientes (a_i, b_i, c_i) na coluna 12 estão dispostos do menor para o maior, sendo que $a_1 = 0$;

Figura 21: Matriz expandida A,$coef_i$, B

$$\begin{bmatrix}
8 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \overline{b_1} & 5 \\
11 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \overline{c_1} & 9 \\
0 & 0 & 121 & 11 & 1 & 0 & 0 & 0 & 0 & 0 & \overline{a_2} & 9 \\
0 & 0 & 225 & 15 & 1 & 0 & 0 & 0 & 0 & 0 & \overline{b_2} & 10 \\
0 & 0 & 0 & 0 & 0 & 225 & 15 & 1 & 0 & 0 & \overline{c_2} & 10 \\
0 & 0 & 0 & 0 & 0 & 324 & 18 & 1 & 0 & 0 & \overline{a_3} & 8 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 324 & 18 & 1 & \overline{b_3} & 8 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 484 & 22 & 1 & \overline{c_3} & 7 \\
1 & 0 & -22 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & \overline{a_4} & 0 \\
0 & 0 & 30 & 1 & 0 & -30 & -1 & 0 & 0 & 0 & \overline{b_4} & 0 \\
0 & 0 & 0 & 0 & 0 & 36 & 1 & 0 & -36 & -1 & 0 & \overline{c_4} & 0
\end{bmatrix}$$

Fonte: AUTOR.(2020)

MN aplicados à Engenharia

Portanto, o polinômio $p(x)$ gerado é:

- O vetor de coeficientes das splines quadráticas é, portanto:
  coef'=[a_1 b_1 c_1 a_2 b_2 c_2 a_3 b_3 c_3 a_4 b_4 c_4]

  coef'=[0.00 1.33 -5.67 **-0.2708 7.2917 -38.4375** 0.06 -2.50 35.00 0.06 -2.75 37.25]

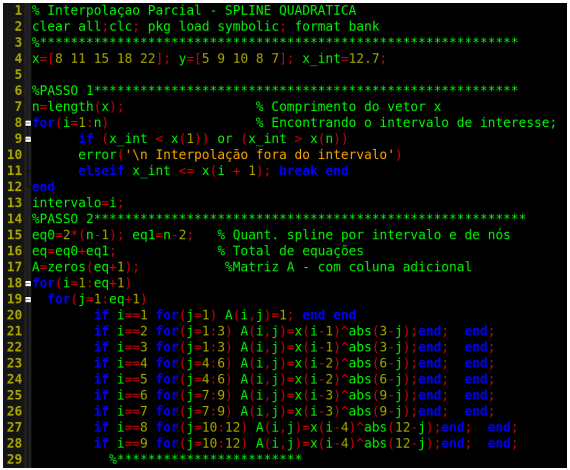
- Desse modo, $f(12,7)$ pertence ao polinômio $f_2(x)$ tal que:

$$f_2(x) = (-0,27)x^2 + 7,29x - 38,43 =$$

$$f_2(12,7) = (-0,2708)(12,7)^2 + 7,2917.(12,7) - 38,4375 = 10,4898$$

MN aplicados à Engenharia

Figura 22: *Script* Spline Linear - parte 01



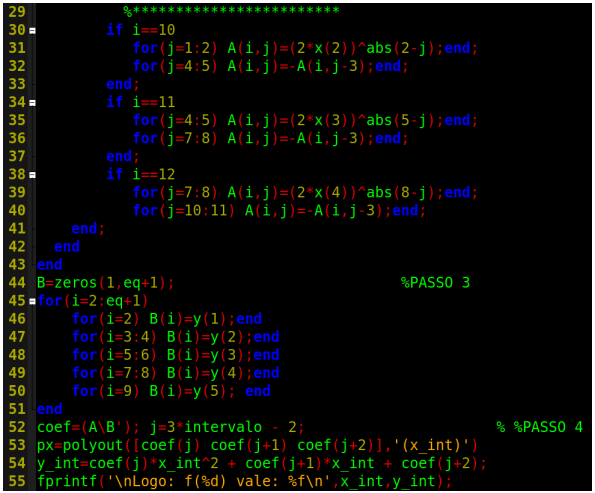
```
% Interpolaçao Parcial - SPLINE QUADRATICA
clear all;clc; pkg load symbolic; format bank
%************************************************************
x=[8 11 15 18 22]; y=[5 9 10 8 7]; x_int=12.7;

%PASSO 1*****************************************************
n=length(x);                  % Comprimento do vetor x
for(i=1:n)                    % Encontrando o intervalo de interesse;
      if (x_int < x(1)) or (x_int > x(n))
      error('\n Interpolação fora do intervalo')
      elseif x_int <= x(i + 1); break end
end
intervalo=i;
%PASSO 2*****************************************************
eq0=2*(n-1); eq1=n-2;   % Quant. spline por intervalo e de nós
eq=eq0+eq1;             % Total de equações
A=zeros(eq+1);           %Matriz A - com coluna adicional
for(i=1:eq+1)
  for(j=1:eq+1)
        if i==1 for(j=1) A(i,j)=1; end end
        if i==2 for(j=1:3) A(i,j)=x(i-1)^abs(3-j);end;  end;
        if i==3 for(j=1:3) A(i,j)=x(i-1)^abs(3-j);end;  end;
        if i==4 for(j=4:6) A(i,j)=x(i-2)^abs(6-j);end;  end;
        if i==5 for(j=4:6) A(i,j)=x(i-2)^abs(6-j);end;  end;
        if i==6 for(j=7:9) A(i,j)=x(i-3)^abs(9-j);end;  end;
        if i==7 for(j=7:9) A(i,j)=x(i-3)^abs(9-j);end;  end;
        if i==8 for(j=10:12) A(i,j)=x(i-4)^abs(12-j);end;  end;
        if i==9 for(j=10:12) A(i,j)=x(i-4)^abs(12-j);end;  end;
          %************************
```

Fonte: AUTOR,(2020)

MN aplicados à Engenharia

Figura 23: *Script* Spline Linear - parte 02



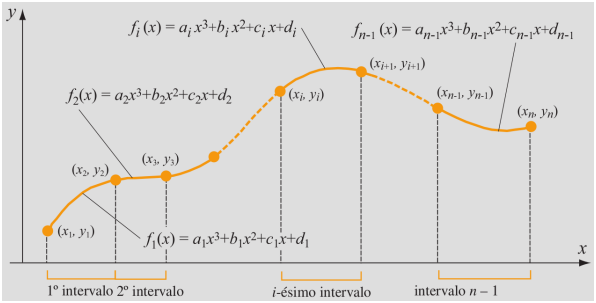
```
29          %**********************
30          if i==10
31             for(j=1:2) A(i,j)=(2*x(2))^abs(2-j);end;
32             for(j=4:5) A(i,j)=-A(i,j-3);end;
33          end;
34          if i==11
35             for(j=4:5) A(i,j)=(2*x(3))^abs(5-j);end;
36             for(j=7:8) A(i,j)=-A(i,j-3);end;
37          end;
38          if i==12
39             for(j=7:8) A(i,j)=(2*x(4))^abs(8-j);end;
40             for(j=10:11) A(i,j)=-A(i,j-3);end;
41       end;
42    end
43 end
44 B=zeros(1,eq+1);                              %PASSO 3
45 for(i=2:eq+1)
46       for(i=2) B(i)=y(1);end
47       for(i=3:4) B(i)=y(2);end
48       for(i=5:6) B(i)=y(3);end
49       for(i=7:8) B(i)=y(4);end
50       for(i=9) B(i)=y(5); end
51 end
52 coef=(A\B'); j=3*intervalo - 2;                          % %PASSO 4
53 px=polyout([coef(j) coef(j+1) coef(j+2)],'(x_int)')
54 y_int=coef(j)*x_int^2 + coef(j+1)*x_int + coef(j+2);
55 fprintf('\nLogo: f(%d) vale: %f\n',x_int,y_int);
```

Fonte: AUTOR,(2020)

MN aplicados à Engenharia

- Em *splines* cúbicas, a interpolação é feita com polinômios de terceira ordem.
- Para um conjunto de dados com n pontos, há $n-1$ intervalos, portanto, um número expressivo de equações, a depender da forma de representação polinomial utilizada (padrão, Lagrange, Newton);

Figura 24: Splines Cúbicas



Fonte: GILAT,(2008)

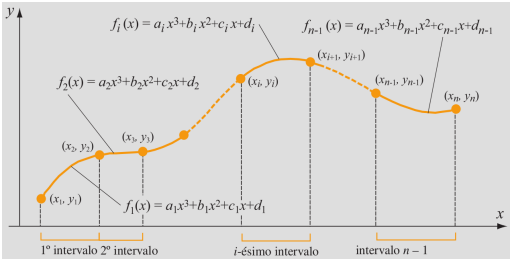
MN aplicados à Engenharia

(1) De forma geral, há $n-1$ equações e, como cada equação tem, *neste caso*, **quatro** coeficientes, um total de $4(n-1) = 4n-4$ coeficientes a serem determinados, tais que:

* Cada polinômio $f_i(x)$ deve passar pelos pontos finais do intervalo, (x_i, y_i) e (x_{i+1}, y_{i+1}), o que significa que $f_i(x_i) = y_i$ e $f_i(x_{i+1}) = y_{i+1}$, portanto:

$$\begin{cases} a_i x_i^3 + b_i x_i^2 + c_i x_i + d_i = y_i \\ a_i x_{i+1}^3 + b_i x_{i+1}^2 + c_i x_{i+1} + d_{i+1} = y_{i+1} \end{cases} \quad \text{para } i = 1, 2, \ldots, n-1$$

Figura 25: Splines Quadráticas



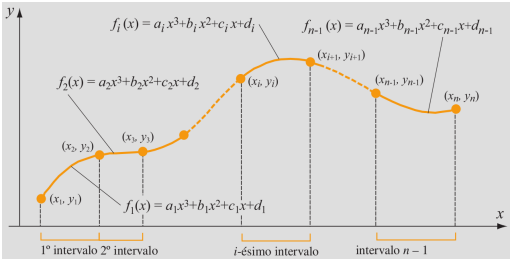
Fonte: GILAT,(2008)

MN aplicados à Engenharia

(2) Nos nós internos, as inclinações (derivadas primeiras) dos polinômios de intervalos adjacentes são iguais, portanto, com a transição da curva que passa por um nó interno de um polinômio para outro:

- A derivada primeira do i-ésimo polinômio é: $f_i'(x) = \dfrac{df_i}{dx} = 3a_i x^2 + 2b_i x + c_i$
- Para n pontos, o primeiro ponto interno é $i = 2$ e o último é $i = n - 1$, o que resulta em: $3a_{i-1}x_i^2 + 2b_{i-1}x_i + c_i = 3a_i x_i^2 + 2b_i x_i + c_i$
- Como há $n - 2$ pontos internos, essa condição fornece $n - 2$ equações.

Figura 26: Splines Quadráticas



Fonte: GILAT,(2008)

MN aplicados à Engenharia

(3) Nos nós internos, as derivadas segundas dos polinômios de intervalos adjacentes devem ser iguais, isso significa que, com a transição da curva que passa por um nó interno de um polinômio para outro, a taxa de inclinação (curvatura) deve ser contínua, portanto,:

- $f_i''(x) = \dfrac{d^2 f_i}{dx^2} = 6a_i x + 2b_i$
- Para n pontos, o primeiro ponto interno é $i = 2$ e o último é $i = n - 1$, o que resulta em: $6a_{i-1}x_i + 2b_{i-1} = 6a_i x_i + 2b_i$
- Como há $n - 2$ pontos internos, essa condição fornece $n - 2$ equações.

- Juntas, as três condições fornecem $4n-6$ equações, entretanto, os $n-1$ *polinômios* têm $4n-4$ *coeficientes*, e com isso duas equações (condições) adicionais são necessárias para que os coeficientes sejam obtidos. As condições geralmente escolhidas assumem que **a derivada segunda seja nula no primeiro e no último ponto**, portanto:
  $6a_1x_1 + 2b_1 = 0$ e $6a_{n-1}x_n + 2b_{n-1} = 0$

* Splines cúbicas com derivadas segundas igualadas a zero nos pontos finais do intervalo são chamadas de **splines cúbicas naturais**.

- A aplicação de todas as condições leva a um sistema de $4n - 4$ **equações com** $4n - 4$ **coeficientes**.

MN aplicados à Engenharia

Splines Cúbicas
Baseadas em polinômios na forma de Lagrange

- A dedução de splines cúbicas utilizando a forma de Lagrange resulta no termo geral:

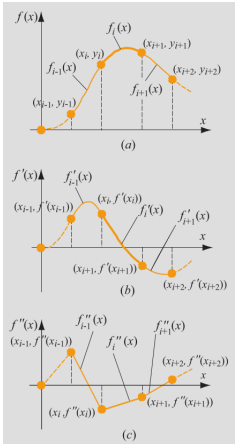
$$f_i(x) = \begin{cases} \frac{a_i}{6h_i}(x_{i+1} - x)^3 + \frac{a_{i+1}}{6h_i}(x - x_i)^3 \\ \quad + \left[\frac{y_i}{h_i} - \frac{a_i h_i}{6}\right](x_{i+1} - x) \\ \quad + \left[\frac{y_{i+1}}{h_i} - \frac{a_{i+1} h_i}{6}\right](x - x_i), \end{cases} ,$$

em que $x_i \leq x \leq x_{i+1}$ e $i = 1, 2, \ldots, n-1$

- E o termo a_i vale:

$$h_i a_i + 2(h_i + h_{i+1})a_{i+1} + h_{i+1}a_{i+2} = 6\left[\frac{y_{i+2} - y_{i+1}}{h_{i+1}} - \frac{y_{i+1} - y_i}{h_i}\right], \text{ para } i = 1, 2, \ldots, n-1.$$

Figura 27: Splines Cúbica



Fonte: GILAT,(2008)

MN aplicados à Engenharia

Ajuste de Curvas e interpolação

Exemplo: Determine as *splines* cúbicas que fazem o ajuste dos dados: $x = [8, 11, 15, 18, 22]; y = [5, 9, 10, 8, 7]$, e encontre $f(12, 7)$.

- **Os cinco pontos geram quatro splines**. A equação cúbica da i-ésima spline é:
  $f_i(x) = \frac{a_i}{6h_i}(x_{i+1} - x)^3 + \frac{a_{i+1}}{6h_i}(x - x_i)^3 + \left[\frac{y_i}{h_i} - \frac{a_i h_i}{6}\right](x_{i+1} - x) + \left[\frac{y_{i+1}}{h_i} - \frac{a_{i+1} h_i}{6}\right](x - x_i)$, para $i = 1, \ldots 4$, em que $h_i = x_{x+1} - x_i$
- As quatro equações contêm cinco coeficientes desconhecidos a_1, a_2, a_3, a_4 e a_5. Nas splines cúbicas naturais, os coeficientes a_1 e a_5 são iguais a zero.
- Os outros três coeficientes são determinados a partir de um sistema linear de três equações, a saber:
  - $h_1 = x_2 - x_1 = 11 - 8 = 3$ e $h_2 = x_3 - x_2 = 15 - 11 = 4$;
  - $h_3 = x_4 - x_3 = 18 - 15 = 3$ e $h_4 = x_5 - x_4 = 22 - 18 = 4$.

MN aplicados à Engenharia

Ajuste de Curvas e interpolação

Splines cúbicas - Exemplo

De modo que tem-se para:

i=1 $h_1 a_1 + 2(h_1 + h_2)a_2 + h_2 a_3 = 6\left[\frac{y_3 - y_2}{h_2} - \frac{y_2 - y_1}{h_1}\right]$

$3.\boxed{0} + 2(3+4)a_2 + 4a_3 = 6\left[\frac{10-9}{4} - \frac{9-5}{3}\right] \rightarrow 14a_2 + 4a_3 = -6,5$

i=2 $h_2 a_2 + 2(h_2 + h_3)a_3 + h_3 a_4 = 6\left[\frac{y_4 - y_3}{h_3} - \frac{y_3 - y_2}{h_2}\right]$

$4.a_2 + 2(3+4)a_3 + 3a_4 = 6\left[\frac{8-10}{3} - \frac{10-9}{4}\right] \rightarrow 4a_2 + 14a_3 + 3a_4 = -5,5$

i=3 $h_3 a_3 + 2(h_3 + h_4)a_4 + h_4 a_5 = 6\left[\frac{y_5 - y_4}{h_4} - \frac{y_4 - y_3}{h_3}\right]$

$3.a_3 + 2(3+4)a_4 + 4.\boxed{0} = 6\left[\frac{7-8}{4} - \frac{8-10}{3}\right] \rightarrow 3a_3 + 14a_4 = 2,5$

- Na forma matricial o sistema resulta em: $\begin{bmatrix} 14 & 4 & 0 \\ 4 & 14 & 3 \\ 0 & 3 & 14 \end{bmatrix} \begin{bmatrix} a_2 \\ a_3 \\ a_4 \end{bmatrix} = \begin{bmatrix} -6,5 \\ -5,5 \\ 2,5 \end{bmatrix}$

Portanto:

$a_1 = 0$; $a_2 = -0,3665$; $a_3 = -0,3421$; $a_4 = 0,2519$ e $a_5 = 0$

MN aplicados à Engenharia

De modo que :

- O polinômio interpolado $p(x) = f(x)$ calculado com a substituição dos coeficientes para o intervalo que contém $x = 12,7$ é $f_2(x)$ com $11 \leq x \leq 15$

  i=1 ...

  i=2 $f_2(x) = \frac{a_2}{6h_2}(x_3 - x)^3 + \frac{a_3}{6h_2}(x - x_2)^3 + \left[\frac{y_2}{h_2} - \frac{a_2 h_2}{6}\right](x_3 - x) + \left[\frac{y_3}{h_2} - \frac{a_3 h_2}{6}\right](x - x_2)$

  $f_2(x) = \frac{-0,3665}{6.4}(15 - 3)^3 + \frac{-0,3421}{6.4}(x - 11)^3 + \left[\frac{9}{4} - \frac{-0,3665.4}{6}\right](15 - x) + \left[\frac{10}{4} - \frac{-0,3421.4}{6}\right](x - 11)$

  $f_2(x) = (-0,01527)(15 - x)^3 + (-0,01427)(x - 11)^3 + 2,494(15 - x) + 2,728(x - 11)$

  i=3 ...

  i=4 ...

MN aplicados à Engenharia

Ajuste de Curvas e interpolação

Portanto,

- $f_2(x) = (-0,01527)(15 - x)^3 + (-0,01427)(x - 11)^3 + 2,494(15 - x) + 2,728(x - 11)$
- $f_2(12,7) = (-0,01527)(15 - 12,7)^3 + (-0,01427)(12,7 - 11)^3 + 2,494(15 - 12,7) + 2,728(12,7 - 11) =$ **10,11**

Comparativo de resultados das splines para $x_{int} = 12,7$

$x = 12,7$	Linear	Quadrática	Cúbica
$f(12,7)$	$9,4250$	$10,4898$	$10,11$

MN aplicados à Engenharia

Ajuste de Curvas e interpolação

Funções auxiliares nativas

- polyfit(*vetor x, vetor y , grau*)
- interp1(*vetor x, vetor y, x interpolado (vetor ou escalar), método*)
  - Interpolação : 'nearest' e 'linear'
  - Interpolação e extrapolação: 'spline' e 'pchip'
  - 'spline' exige que o intervalos estejam equidistantes para mitigar erros.

Referências

- ÁVILA, Sérgio Luciano. **Cálculo numérico aplicado à engenharia elétrica com MATLAB**.
- COSTA, Jonatha R. **Notas de aula: Métodos Numéricos**. Disponível em: `https://github.com/jonathacosta/NM`.
- COSTA, Jonatha R. **Códigos em Python**. Disponível em: `https://github.com/jonathacosta/NM`.
- GILAT, Amos. **MATLAB com aplicações em engenharia**. Bookman, 2009.
- IFCE. **PUD – Programa da Unidade Didática**. Bacharelado em Controle e Automação, *campus* Maracanaú.